

Logică digitală

Circuite secvențiale

Universitatea Politehnica Timișoara
Facultatea de Automatică și Calculatoare
Departamentul de Calculatoare și Tehnologia Informației

April 7, 2024

Cuprins

1	Scopul lucrărilor	3
2	Circuitul Flip-Flop de tip D	4
3	Registrul cu încărcare paralelă și semnal de validare al intrărilor	6
4	Modelarea circuitelor secvențiale folosind 2 segmente	7
5	Proiectarea circuitelor de tip numărător	8
6	Proiectarea automatelor cu stări finite	10

1 Scopul lucrărilor

Scopul acestei părți a laboratorului este familiarizarea studenților cu conceptele de circuit secvențial, flip-flopuri, registre cu încărcare paralelă, numărătoare și automate cu stări finite. În urma acestor lucrări studenții vor putea:

- să definească conceptul de circuit secvențial și să diferențieze aceste circuite de cele combinatoriale
- să utilizeze circuite flip-flop de tip D în realizarea de circuite mai complexe
- să proiecteze un numărător sincron ce utilizează flip-flopuri de tip D.
- să realizeze tabelele de tranziții, codificarea stărilor, tabelul de ieșire al unui automat cu stări finite.
- să proiecteze un automat cu stări finite pe baza unui enunț sau diagrama de tranziții date
- să modeleze și să simuleze în limbajul verilog sub-circuitele aferente realizării circuitului propus.

În continuare ne propunem să realizăm un cronometru digital. Schema bloc a circuitului propus este redată în Figura 1.

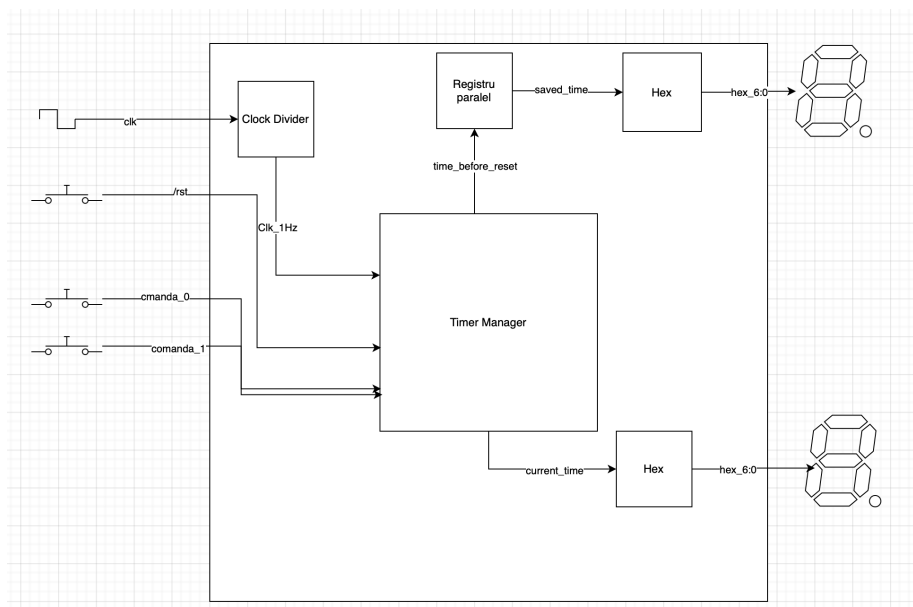


Figure 1: Schema bloc a cronometrului digital propus

Pe scurt modulele ce vor fi realizate în cadrul lucrărilor de circuite secvențiale au următoarele specificații:

- Clock_Divider - circuit ce are ca rol generarea unui semnal de tact cu o frecvență de 1 Hz. Acesta va primi ca intrare semnalul de tact de la FPGA și va realiza divizarea acestuia.

- Registrul paralel - circuit cu intrare paralelă și ieșire paralelă, ambele pe 4 biți capabil să rețină o valoare.
- Hex - circuit combinațional de tip Decodificator, responsabil cu convertirea unui număr din binar în reprezentare 7 segmente
- Timer Manager - Modul principal de control al cronometrului. Primește ca și intrare o comanda de la butoane de tip start, stop, reset și are ca rol să pornească, oprească sau să reseteze valoarea cronometrului.
- Cronometru - modulul Top level entity care va fi modelat și încărcat pe FPGA.

Detalierea modulelor componente se va realiza la fiecare Lucrare. Veți avea mereu în viziune aplicația de Cronometru și veți completa modulele prezentate mai sus în mod gradual. În final vor exista 5 surse verilog.

2 Circuitul Flip-Flop de tip D

Principalul element secvențial folosit în circuitele digitale este bistabilul sincron activ pe frontal semnalului de tact (edge triggered flip-flop). În cazul acestor bistabile, comutarea este posibilă doar pe unul dintre fronturile semnalului de tact; spre deosebire de bistabilul activ pe palierul semnalului de tact, edge triggered flip-flop prezintă avantajul de a avea ieșirea stabilă pe întreaga durată a semnalului de tact, nefiind afectată de eventualele glitch-uri ce pot apărea la intrare.

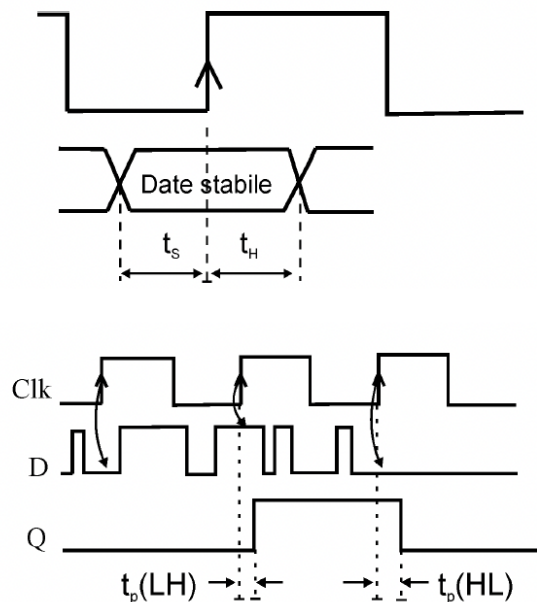


Figure 2: Timpii de setup (t_s), hold (t_h) și de întârziere (t_{pLH} , t_{pHL}) aferenți bistabilelor de tip D active pe front

În ceea ce privește funcționarea acestor dispozitive, există o serie de timpi legați de funcționarea FF-urilor, care trebuie luați în calcul și respectați în cazul proiectării circuitelor reale:

- Timpul de setup: se referă la timpul necesar pentru ca semnalul de intrare (D) să rămână stabil înainte de apariția frontului semnalului de tact;
- Timpul de hold: reprezintă timpul în care datele de intrare nu pot fi modificate după apariția frontului semnalului de tact în vederea încărcării corecte a acestora;
- Timpul aferent întârzierii datorate propagării (t_p): constituie timpul necesar basculării FF-ului (clock to Q delay)

În Figura 3 este prezentat bistabilul de tip D cu semnal de validare (EN) a intrărilor.

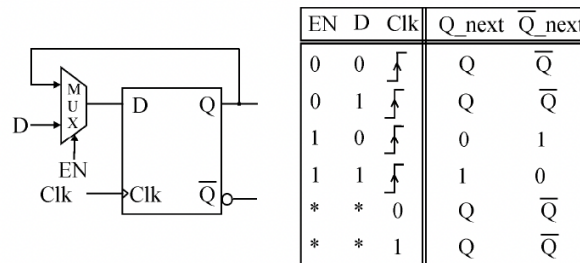


Figure 3: Flip-Flop (FF) de tip D cu semnal de validare - schema bloc și tabelul stărilor

Mai jos este prezentat codul Verilor de modelare comportamentală a bistabilului de tip D.

```

module D_FF_EN
    (input clk,
     input rst,
     input d,
     input en,
     output reg q);

    always @(posedge clk or posedge rst)
    begin
        if (rst)
            q <= 0;
        else if (en)
            q <= d;
        end
    endmodule

```

Cerințe specifice:

- Pornind de la modelarea bistabilului D, realizați simularea aferentă acestuia.

3 Registrul cu încărcare paralelă și semnal de validare al intrărilor

În sistemele de calcul este necesară memorarea datelor pe n biți. Acest lucru poate fi realizat în două moduri: paralel – când toți biții sunt transmiși concomitant pe n linii, sau serial când biții sunt transmiși pe rând – bit după bit. Pentru prima situație este nevoie de un ciclu de tact pentru a încărca informația, iar pentru cea de-a doua sunt necesari n cicli de tact pentru acest lucru. De asemenea din punct de vedere al modului prin care sunt accesați cei n biți memorati, sunt 2 variante: serie – biții sunt citați pe rând în n cicluri de tact, sau paralel - toți biții sunt citați concomitant în același ciclu de tact.

În Figura 4 sunt prezentate toate versiunile de regiștri ce se pot regăsi în sistemele de calcul.

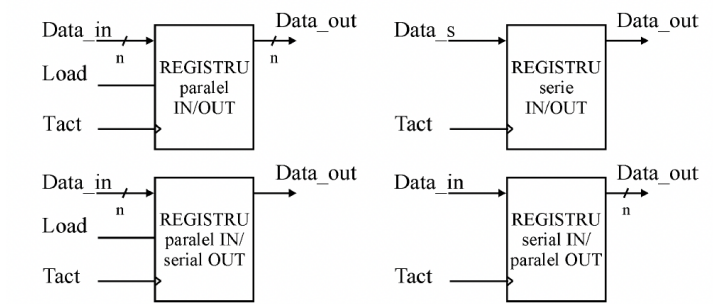


Figure 4: Regiștrii cu încărcare paralelă/serie și ieșire paralelă/serie

Cea mai simplă variantă de registru este registrul cu încărcare paralelă și citire a datelor paralelă. Pentru a stabili momentul când se încarcă informație nouă, este prevăzut un semnal de validare a intrărilor (LD). Schema pentru acest registru este prezentată în Figura 5.

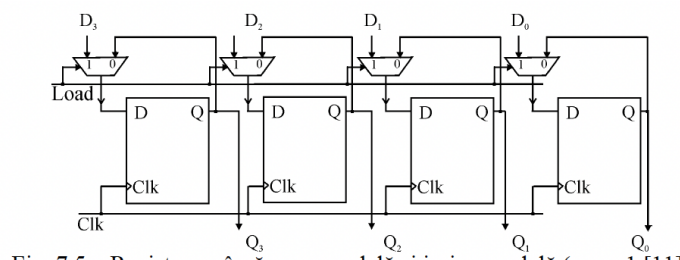


Figure 5: Registrul cu încărcare paralelă și ieșire paralelă

Codul Verilog aferent acestui registru este următorul:

```
module paralel_register
    (input clk, rst,
     input load,
     input [3:0]d,
     output reg [3:0] q);
    always @( posedge clk or posedge rst)
```

```

begin
    if (! rst)
        q <= 0;
    else
        if (load == 1'b1)
            q <= d;
    end
endmodule

```

Așa cum se poate observa din figură, semnalul de încărcare Load este activ pe 1 logic. Acesta determină activarea corespunzătoare a căii din multiplexor care selectează la ieșirea acestuia valoarea semnalului de intrare date (Di).

Cerințe specifice:

- Realizați un modul de simulare în Verilog pentru a verifica funcționarea corectă a registrului propus în codul de mai sus.
- Vom utiliza modulul nou creat pentru a memora valoarea cronometrului la un anumit moment. În acest sens realizați un modul verilog Cronometru.v care va fi modulul Top Level al întregii aplicații. Toate celelalte module definite vor fi instanțiate în acest modul. Astfel, modulul Cronometru va avea o modelare pur structurală.
- Integrați registrul paralel în Cronometru.v și verificați corectitudinea lui prin conectarea ieșirii la un circuit afișaj cu 7 segmente. În acest moment modulul Cronometru va avea ca intrări un număr pe 4 biți, semnalul de tact, reset și cel de Load. Ieșirea acestui modul va fi direct conectată la ieșirea modulului afișaj cu 7 segmente, fiind astfel o ieșire pe 7 biți.

4 Modelarea circuitelor secvențiale folosind 2 segmente

Pentru circuite mai complexe, care conțin o parte combinațională și o parte secvențială, este util să separăm partea de memorare a datelor (secvențială), de cea care calculează noile date care vor fi memorate (combinațională). Pentru aceasta folosim o construcție cu două segmente (două blocuri always): unul pentru partea combinațională și unul aferent părții secvențiale.

Codul Verilog de mai jos, reprezintă un template uzual utilizat pentru a realiza modelarea cu 2 segmente.

```

module <module_name> #(list_of_parameters)(input <input_var>, output <output_var>);

// declarare semnale interne, le vom denumi mai jos ca stare viitoare "_nxt" si stare
// curenta "_reg"
reg [<list_of_parameters "width">:0] <ceva_var>_reg, <ceva_var>_nxt;

// blocurile assign si always se executa in paralel
// logica combinationala - folositi doar "=" blocking
always @* begin
    // toate starile viitoare primesc starea curenta
    <ceva_var>_nxt = <ceva_var>_reg;

    // in functie de starea curenta (_reg) sau de input-uri(input_var) calculam starea
    // viitoare
    if(<ceva_var>_reg | input_var) begin

```

```

        <ceva_var>_nxt = <ceva_var>_reg;
    end
end

// blocurile assign si always se executa in paralel
// logica secventiala - folositi doar "<=" non-blocking
always @ (posedge clk or negedge rst) begin
    // daca reset-ul este activ atribuim valori default pentru starile curente
    if(rst) begin
        <ceva_var>_reg <= 'b0;

    end else begin // daca reset-ul nu este activ facem update la stari
        <ceva_var>_reg <= <ceva_var>_nxt;
    end
end

// blocurile assign si always se executa in paralel
// continuous assign
// assign <wire_variable> = <reg_variable>
assign <output_var> = <ceva_var>_reg;

endmodule

```

5 Proiectarea circuitelor de tip numărător

În acest capitol, încercăm construcția unui numărător pe 4 biți, care numără crescător până la atingerea valorii 15 (F în baza hexazecimală). Cele 2 părți pot fi cu ușurință dinse în Figura 6 : element de memorare de tip registru cu încărcare paralelă, și element combinațional de procesare - sumatorul.

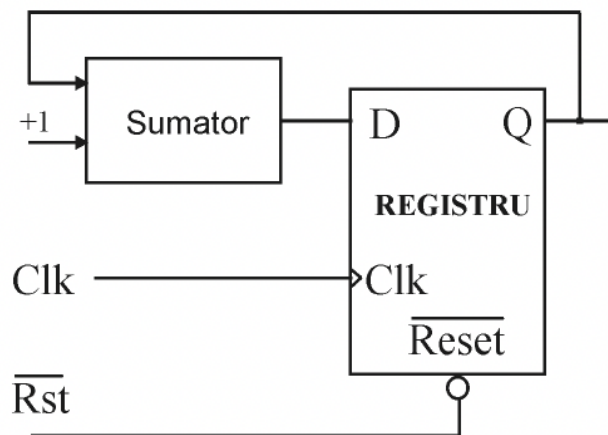


Figure 6: Schema bloc a unui numărător

Se impun următoarele precizări:

- Primul always block este folosit pentru a descrie partea secvențială – registrul în care este memorată starea numărătorului. În lista sa de sensibilități sunt prezente:
 - Tactul – frontul crescător;

– Resetul – front descrescător;

Ca și constrângere, din punctul de vedere al sintaxei, nu pot coexista în lista de sensibilități atât frontul cât și palierul unui semnal. Așadar, dacă este specificat reset-ul acesta trebuie să fie specificat tot ca și front.

- Al doilea always block este folosit pentru a descrie partea combinațională—pregătește următoarea stare a numărătorului. În lista sa de sensibilități sunt prezente toate semnalele a căror modificare atrage după sine modificarea valorii noii stări, deci re-evaluarea codului dintre begin și end al blocului always.

Codul Verilog HDL este prezentat în cele ce urmează.

```
module numarator
  (input clk, rst
   output [3:0] q);

  reg [3:0] q_nxt, q_reg;

  //always block for sequential part
  always @(posedge clk or posedge rst)
    begin
      if (rst)
        q_reg <= 0;
      else
        q_reg <= q_nxt;
    end

  //always block for combinational part
  always @(q_reg)
    begin
      q_nxt = q_reg + 1'b1;
    end
  assign q = q_reg;
endmodule
```

Cerințe specifice:

- Pornind de la definiția unui numărător crescător prezentată mai sus, realizați schema hardware a acestuia utilizând circuite FF de tip D și porți.
- Realizați un modul de simulare în Verilog pentru a verifica funcționarea corectă a numărătorului propus în codul de mai sus.
- Utilizând informațiile precizate, realizați un modul de divizare a frecvenței de tact a FPGA-ului pentru a avea un semnal de tact cu frecvența de un Hz utilizând conceptul de numărător definit.
- Realizați un al doilea modul numărător (ce va fi extins în următoarea lucrare) ce va avea ca input noul semnal de tact divizat, va incrementa un număr de secunde, și va avea ca ieșire numărul de secunde curent.
- Integrați noul divizor de tact, respectiv noul modul ce numără secunde în Cronometru.v și verificați corectitudinea lui prin conectarea ieșirii la circuitul registru deja prezent în Cronometru.v. Pentru simplitate, setați intrarea de Load a registrului ca fiind activă.

6 Proiectarea automatelor cu stări finite

Se dorește realizarea unei unități de control pentru un automat de cafea care să funcționeze în felul următor:

- 1 Automatul funcționează cu monede de 0.5 lei, respectiv bancnote de 1 leu.
- 2 Există trei variante de băuturi: espresso, espresso lung și cappuccino
- 3 Fiecare băutură costă 2 lei.
- 4 Selecția uneia dintre cele 3 băuturi se poate face după ce s-a introdus suma de 2 lei.
- 5 Automatul nu dă rest; în cazul introducerii unei sume mai mari de 2 lei, automatul va permite selecția unei singure băuturi.

Pentru modelarea unității de control pentru problema descrisă mai sus, va fi folosit un circuit digital ce va avea 4 semnale de intrare și 3 semnale de ieșire.

Semnalele de intrare au următoarea semnificație:

- 1 Două semnale de intrare (Credit05 și Credit10) pentru cele două modalități de introducere a banilor; valoarea 1 aferentă acestor două semnale indică faptul că a fost introdusă una dintre cele două.
- 2 Două semnale de intrare aferente celor 3 băuturi (coffee[1:0]): valoarea 00 reprezintă niciun fel de selecție a băuturii, valoarea 01 este folosită pentru espresso, 10 pentru espresso lung, iar 11 pentru cappuccino. Fiecărui tip de cafea i se asociază un semnal de ieșire.

Modelarea unui astfel de automat cu stări finite necesită ca prim pas realizarea diagramei de stări aferente. Pentru unitatea de control descrisă anterior se va folosi un automat de tip Moore. Diagrama de stări asociată acestui automat este prezentată Figura 7.

Automatul cu stări finite din Figura 7 prezintă un număr de 8 stări. Figurile 8, 9 și 10 prezintă codificarea stărilor, tabelul de tranziții asociat acestui automat, precum și tabelul aferent ieșirilor. Fiind un automat de tip Moore, ieșirile sunt dependente doar de starea automatului, nu și de intrările acestuia.

Automatul cu stări finite va conține un registru de stare de 3 biți. Pe baza tabelului 8.2 se pot deriva ecuațiile logice pentru fiecare din cele 3 bistabile din cadrul registrului de stare, iar pe baza tabelului 8.3 se pot determina ecuațiile logice pentru cele 3 ieșiri.

Codul de mai jos reprezintă o parte din modelarea automatului.

```
'define INIT 3'b000
'define C05 3'b001
'define C10 3'b010
// DE COMPLETAT cu definirea stărilor

module coffee_fsm(
    input clk, rst,
    input credit05, credit10,
    input [1:0] coffee,
    output [2:0] current_state
    output expr, expr_l, capp);
```

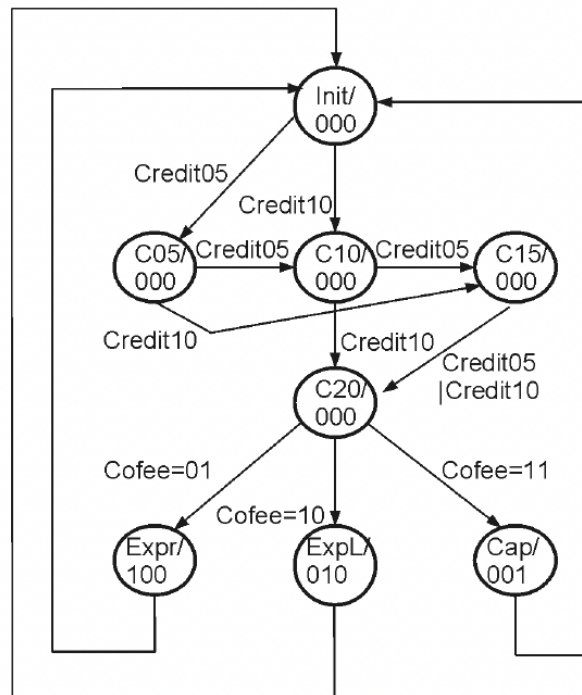


Figure 7: Diagrama de stări Moore a automatului detaliat

Stare	Codificare		
	Q2	Q1	Q0
Init	0	0	0
C05	0	0	1
C10	0	1	0
C15	0	1	1
C20	1	0	0
Expr	1	0	1
ExpL	1	1	0
Cap	1	1	1

Figure 8: Codificarea stărilor

Stare curentă			Intrări				Stare viitoare		
Q2	Q1	Q0	credit05	credit10	coffee[1]	coffee[0]	D2	D1	D0
0	0	0	0	0	x	x	0	0	0
0	0	0	0	1	x	x	0	1	0
0	0	0	1	0	x	x	0	0	1
0	0	1	0	0	x	x	0	0	1
0	0	1	0	1	x	x	0	1	1
0	0	1	1	0	x	x	0	1	0
0	1	0	0	0	x	x	0	1	0
0	1	0	0	1	x	x	1	0	0
0	1	0	1	0	x	x	0	1	1
...	...	...	...	...	...	...	...	...	...

Figure 9: Tabelul de tranziții al automatului

Stare curentă			Ieșiri		
Q3	Q2	Q1	exprr	expr_l	capp
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	1	0	0
1	1	0	0	1	0
1	1	1	0	0	1

Figure 10: Tabelul ieșirilor automatului

```

reg [2:0] state_nxt, state_reg; //starea urm a registrului de stare,
respectiv starea curenta
reg expr_reg, expr_nxt, expr_l_reg, expr_l_nxt, capp_reg, capp_nxt;
//segmentul secvential memoreaza starea curenta
always @( posedge clk or posedge rst)
begin
    //daca resetul este activ (0 logic)
    if (rst)
        begin
            state_reg <= 0; //q_reg denota starea curenta
            expr_reg <= 0;
            expr_l_reg <= 0;
            capp_reg <= 0;
        end
    else
        begin
            state_reg <= state_nxt;
            expr_reg <= expr_nxt;
            expr_l_reg <= expr_l_nxt;
            capp_reg <= capp_nxt;
        end
    end
//segmentul combinational pregateste starea urmatoare, care va fi incarcata
//in registru de stare
always @( state_reg, coffee, credit05, credit10) //lista de senzitivitati contine
toate semnalele a caror modificare
begin //produce re-evaluarea codului dintre begin ... end
    state_nxt = state_reg;
    expr_nxt = 0;
    expr_l_nxt = 0;
    capp_nxt = 0;
    //descrierea automatului
    case (state_reg)
        'INIT : begin
            state_nxt = 'INIT;
            expr_nxt = 0;
            expr_l_nxt = 0;
            capp_nxt = 0;
            if(credit05)
                state_nxt = 'C05;
            if(credit10)
                state_nxt = 'C10;
        end
        // DE COMPLETAT restul starilor
    endcase
end
assign state = state_reg;
// DE COMPLETAT restul iesirilor

```

Cerințe specifice:

- Pornind de la tabelul de tranziții completați restul de linii și realizați schema hardware a acestui automat circuite FF de tip D și porți.
- Realzați un modul de simulare în Verilog pentru a verifica funcționarea corectă a automatului propus în codul de mai sus. A se completa codul pentru a avea o modelare completă.
- Utilizând informațiile precizate, extindeți modulul de numărare realizat în lucrarea precedentă astfel:

- circuitul va primi ca intrare o comandă pe 2 biți cu următoarele specificații: 00 - Idle, 01 - start, 11 - stop, 10 - reset.
- circuitul va utiliza semnalul de tact de 1Hz adus la intrare de la divizorul de semnal.
- dacă nu se primește nici o comandă (valoarea intrării comandă este 00) circuitul așteaptă.
- dacă se primește comanda Start circuitul începe să numere.
- dacă se primește comanda Stop circuitul se oprește din numărare și poate să reia numărarea prin Start, sau să reseteze valoarea la 0 prin Reset.
- dacă se primește comanda Reset, circuitul va salva ultima valoare în registrul paralel și va reveni la starea de Idle.
- Circuitul va afișa pe ecran numărul de secunde la care a ajuns pe un afișaj cu 7 segmente. Instanțiați în modulul Cronometru un nou afișaj 7 segmente diferit de cel utilizat pentru registrul paralel.
- Integrați noul modul în Cronometru.v și verificați corectitudinea lui. Scopul final este acela de a avea un cronometru complet funcțional care să realizeze funcțiile de start, stop, reset și să afișeze pe un modul 7 segmente timpul curent, respectiv pe un al doilea modul 7 segmente valoarea salvată în registrul paralel după comanda reset.

Cerințe extra bonus:

- Extindeți cronometrul să utilizeze 4 afișaje cu 7 segmente pentru a afișa timpul ca min:sec.
- Realizați un circuit combinațional de conversie a timpului din hexadecimale în BCD în așa fel încât valoarea 0x0A să devină 10. Noul modul va fi inserat între modulul Timer Manager, respectiv Registrul paralel, și modulele Hex.

Bibliografie

- [1] O. Boncalo, A. Amăricăi. "Proiectarea circuitelor digitale folosind Verilog HDL – Analiza și Sinteză". Editura Politehnica, 2011.
- [2] J. F. Wakerly – Digital Design: Principles and Practices, 3rd Edition, Prentice Hall, 2000
- [3] J. Bhasker - A Verilog HDL Primer, Third Edition - Star Galaxy Publishing, 2005
- [4] R. Haskell, D. Hanna - Introduction to Digital Design Using Digilent FPGA Boards – Block Diagram/Verilog Examples – LBE Books, 2009
- [5] S. Brown, Z. Vrsanec - Fundamentals of Digital Logic with Verilog Design - McGraw-Hill, 2007
- [6] De10-Lite board User Manual https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/Boards/DE10-Lite/DE10_Lite_User_Manual.pdf